社団法人 電子情報通信学会
THE INSTITUTE OF ELECTRONICS,
INFORMATION AND COMMUNICATION ENGINEERS

信学技報
TECHNICAL REPORT OF IEICE.
NLC 2001-43 (2001-10)

帰納的学習による言い換え表現を含む質問文生成規則の獲得

村山 耕一　　荒木 健治　　栃内 香次

北海道大学大学院工学研究科
〒060-8628 札幌市北区北 13 条西 8 丁目

E-mail: {kou, araki, tochinai}@media. eng. hokudai. ac. jp

あらまし

ユーザとシステムの対話を想定した場合，システムはユーザの問い合わせに対して協調的な応答を返すだけではなく，ユーザに対して質問を行うことが必要になってくると考えられる．また，自然な対話という観点から，システムは常に一つの表現のみで質問を行なうのではなく，状況により多種多様な表現で質問を行なうべきである．そこで本研究ではこれらの点をふまえ，言い換えを含む質問文生成規則の獲得の研究を行ってきた．本システムでは複数の同じ内容を表す質問文から質問文の生成規則と言い換え規則を帰納的に学習する．評価実験を行なった結果，質問文生成規則と言い換え規則を同時に獲得していく様子が確認された．

キーワード　質問，言い換え，帰納的学習，対話処理，自然言語処理

Acquiring Rules of Generation of Query Expressions with Paraphrases Using Inductive Learning

Koichi MURAYAMA,　Kenji ARAKI,　and　Koji TOCHINAI

Graduate School of Engineering, Hokkaido University
N13-W8, Kita-ku, Sapporo, 060-8628, Japan

E-mail: {kou,araki,tochinai}@maedia.eng.hokudai.ac.jp

Abstract

In this paper, we have described the learning method which acquires rules of generation of query expression with paraphrases. In the dialog between a user and a system, we have two problems. One is that only cooperative replying by the system does not mean completely reply, and the other is that only one query expression does not deal with various situations. To resolve these problems, we proposed a method of acquiring rules of generation of the query expression with paraphrases using the inductive learning. In this method, the system acquires the rules by comparing query expressions which express same situation. From the results of the evaluation experiment, we confirmed the effectiveness of this method.

Key words　query, paraphrase, inductive learning, natural language processing

1. はじめに

効率的な対話システムを実現するためには，ユーザの問い合わせに対して，システムが適切に応答を行うだけでは不十分である．例えば，ユーザの問い合わせに対し，ユーザの発話のみではシステムが応答するための情報が不足している場合には，逆にシステムがその不足した情報を得るためにユーザに対して質問を行なうような場合も必要になってくる[1]．そこで我々は質問応答のこの点に注目し，ユーザの発話もしくは発話から導かれる事柄に対しシステムが不足した状況を補うために質問を行ない，ユーザから情報を獲得する手法の開発を行ってきた[2]．

対話処理を行う古典的なシステムに ELIZA[3]や SHRDLU[4]があるが，ELIZA に代表される対話処理システムでは，ユーザの入力した文をあらかじめ用意されているパターンと比較しそのパターンに応じてシステムが応答を行う，という仕組みになっている．ここでこのようなシステムを質問文の生成に適用した場合，ユーザの入力によってのみシステムの行なう質問が決定されるため，システムは質問をすることができてもどのような情報を入手するための質問なのかということを管理することはできない．本研究において，質問とは情報を得るための手段と考えているので，情報の管理に適さないこれらの質問応答システムの処理をそのまま本研究に適用してシステムを構築することは行わない．この問題を解消するために本研究ではユーザの問い合わせとシステムの応答の間に「状況」を表す構造を導入し質問の生成を行なっている．ここで扱う状況とは状況意味論[5]で扱う状況とは異なり，与えられた文を意味表現で表したものを指す．また，SHRDLU では積み木の世界に限定して会話を行うことには成功しているが，本研究のように任意の状況に対して質問を行なうようなシステムには適用が難しい．

一方で，自然な対話を行うという点においてユーザの問い合わせに対しシステムが常に一つの表現のみで質問(応答)を行なうことは対話に不自然さをもたらす一因であると考えられる．対話に自然性を持たせるためには，話す相手や与えられたタスクなどの状況によってそれに応じた多種多様な表現で質問を行なわなければならない．そこで本研究では質問生成にこれらの点を加え，言い換え規則を含む質問文生成規則の獲得を行っている．

また，近年，言い換えに関する知識獲得の研究[6],[7]が盛んに行われている．しかし，これらの研究の中で対話処理にまで適用範囲を広げたものは少なく，本研究が目指している質問文生成に関する言い換え規則の獲得，取り分け，情報を獲得するという側面は考慮されていない場合が多い．この点からも本研究の意義は十分にあると考えられる．

なお，本研究において我々は質問に用いられる表現はどのような場面でも質問内容が等しければ同様の構造を持っているはずで，その表現を限定するのが文脈やタスクの要素であると考え，本研究でもこの点をふまえて対話システムの目的に限定されない質問文生成規則の獲得を目指している．

以下，2 章では状況を用いる必要性について述べ，3 章では本システムが扱う意味表現および規則獲得の原理について述べる．4 章では具体例を交えシステムの処理を説明し，5 章，6 章でそれぞれ実験の結果と考察について述べる．

2. 状況と質問生成

2.1 単純なパターン処理による質問生成

我々はこれまでに，ユーザの入力した文とシステムが生成すべき質問文を組とするデータから，システムが質問文を生成する規則を獲得していくシステムの研究を行ってきた[8]．このシステムは ELIZA に代表される典型的なパターン処理を用いて対話を行うシステムにおけるパターン部分を，ユーザの入力とシステムの質問を組として帰納的学習により獲得していくシステムであった．

ここで，例えば次のような対話においてユーザの入力に対してどのようにシステムが質問の生成を行なうかを考えてみる．

(a)ユーザ＞会議室を予約したいのですが．
(b)システム＞予約する時間をお願いします．
(c)ユーザ＞9 日の午後 6 時から予約したいのですが．
(d)システム＞どこの会議室にしますか？

上の例では，最初のユーザの入力(a)から会議室の予約を行なうことがわかるので，システムはユーザに対して予約に必要な情報を尋ねている．始めに(b)の質問で予約の時間を尋ね，その次に(d)の質問でどの会議室にするかを尋ねている．ここで，前述のシステムを用いて質問生成規則の学習を行なうと，(a)と(b)，(c)と(d)の組み合わせで学習処理が行なわれるが，(d)の質問は(c)の入力に対して行なわれたと考えるのではなく，(a)で決定されたタスク(ここでは会議室の予約)に必要な質問を行っていると考えるのが自然である．このことから，入力に対応するパターンを用いて質問を生成するだけでは知的な対話を行うことが難しいと云える．

本研究ではこの点を解消するために，次節で述べる「状況」を導入することにより，知的な質問生成シス

テムの構築を目指している．

2.2 状況を用いた質問生成

前節で述べたように，ユーザの入力に対して直接応答を返すのではなく，ユーザの入力とそれ以前の対話からどんな情報を獲得するべきかを判断し，その情報を得るためにシステムが質問を行なうと考えた方がより人間に近い処理だと考えられる．このことから，本研究ではユーザの入力に対して直接質問を行なうのではなく，ユーザの入力から導かれる状況（ここでは，会議室を使用したい）に対して質問を行なうことにしている．

以上のようなことから，本研究ではシステムが質問を行なう過程を次のように考える．

1)ユーザの入力もしくはそれまでの会話から，会話の状況が決定される．
2)決定された状況にマッチした質問を選択する．
3)規則にしたがって質問を生成する．

このようにユーザの入力から状況を導く処理と質問を行なう処理を別々に扱うことにより，質問生成の問題と対話における焦点移動のような文脈処理の問題とに分けることができるため，文脈やタスクに左右されない汎用的な質問表現を獲得することができると考えられる．現在，本システムでは質問生成の問題のみを扱っているが，今後研究が進むにつれ，会話から状況を導く処理をシステムに組み込んでいきたいと考えている．

3. 質問文生成規則

3.1 意味表現

本システムではより柔軟な制御を行うために，質問文と応答文を表層表現のまま扱わずに，表層表現を格フレームから成る意味表現に変換したものを使用する．このことは，今後，他の研究分野において意味解析ツールの開発が十分期待されうることを考慮すれば，本システムが意味表現をあつかうことに問題はないと考えられる[9]．

本研究で扱う意味表現は，スロットとスロット値からなる一つ以上のフレームにより構成される．また，意味表現は(株)日本電子化辞書研究所のEDR電子化辞書における日本語コーパス内の意味構造[10]を参考にした．

以下に例を示す．

・いつ太郎は札幌に帰ってきた？

[[main 帰ってくる][time いつ][goal 札幌][agent 太郎] [atb past]]

この例では，主となる概念（文の述語となっている概念など）を示すスロットmain，事象の起こる時間を示すtime，事象の主体または対象の最後の位置を示すgoal，有意志動作を引き起こす主体agent，話者の視点を表すatbから構成されている．

3.2 質問文生成規則の獲得

本システムでは前節で述べた意味表現を用いて質問文の生成および生成規則の獲得を行なっている．2章で述べたように，本システムはユーザの入力から導き出された状況に対して質問を行なうシステムであるが，質問文生成規則を獲得する時点では質問文の組を与えるだけで質問を行なう状況は与えていない．

その理由は，意味構造の特徴として，ある状況が与えられたときその状況を表す文の main スロットに対してスロット値が埋まらないスロットが存在する場合，そのスロットの値に適切な質問表現を代入することで与えられた意味表現（ここでは状況）に対する質問文を生成することができるからである．これを逆に捉えると，ある質問文が与えられたときその質問文から質問表現を抽出できれば，状況を導くことができると考えられる．

本システムではこのことを利用して質問文で構成されるデータから質問文生成規則を学習し，与えられる状況に対して質問文を生成する．

4. システム構成

本章では本システムの構成および処理内容とシステム稼動による動作例を述べる．各処理部は実験の結果をもとに，今後さらに改良される予定である．

4.1 構成

本システムにおける各処理部の関係を図1に示す．システムの動作は学習時と応答生成時の二つに大きく分けられる．

学習時には，まず同じ内容を表す一つ以上の質問文が学習データとして学習処理部に入力され，質問表現辞書を参照しながら与えられたデータから言い換え規則と質問文生成規則を獲得する．獲得された規則はルール辞書に記述される．

一方，応答生成時には，ユーザから与えられた入力文をあてはめることができる構造をもつ規則をルール辞書から検索し，その規則を用いて応答文（ここでは質問文）を生成する．次に，生成された応答文の正誤判定をユーザが行い，その結果を用いてルール辞書および質問表現辞書の更新を行う．

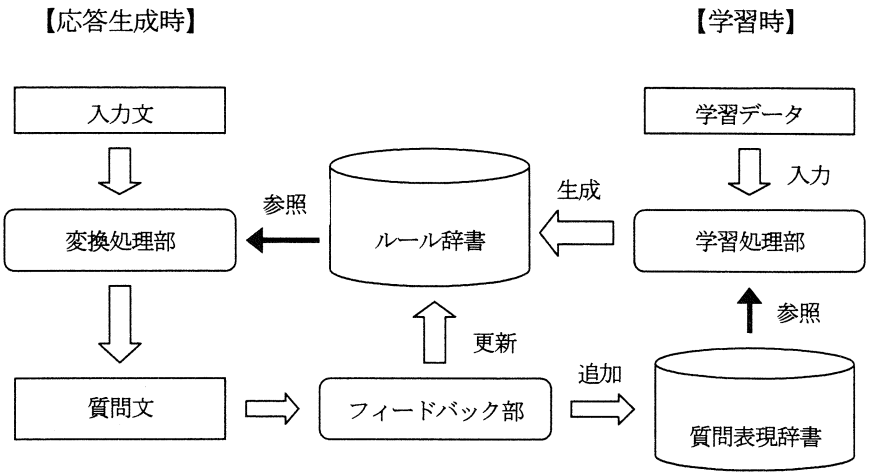


図1　システム処理構成

4.2 質問表現辞書

本システムでは学習処理時に，同一内容を表す質問文の各共通部分を変数に置き換えることで一般化処理を行い，質問生成のための規則および言い換え規則を獲得している．ここで学習データには同一内容を表す複数の質問文を用いているので，共通部分には質問表現が含まれていることがほとんどである．このようなデータからそのまま学習処理を行ってしまうと，質問表現が一般化されてしまい質問文を生成できなくなってしまう．例えば，「いつ帰ってきましたか？」と「いつ戻ってきましたか？」というデータが与えられた場合，「いつ」が共通部分なので「@，帰ってきましたか？」「@，戻ってきましたか？」（@は変数）という規則が獲得され質問表現が失われてしまう．この場合，入力データをそのまま言い換え規則として獲得すべきであり「いつ」が一般化されてはいけない．これを防ぐには，一般化されてはいけない質問表現等の知識をシステムが知っている必要がある．

そのために本システムでは，質問表現辞書を用意し，上記例「いつ」のような一般化されてはいけない語を辞書に記述している．質問表現辞書は学習処理部が規則を獲得する際に参照される．また，質問表現辞書は学習開始時に空の状態であり，応答生成時のフィードバック処理により，辞書の更新が行われる．

次に質問表現辞書の例を示す．

(1)

[[time いつ]]（いつ）

[[time [[main 時][mod 何]]]]（何時に）

4.3 学習処理部

同じ内容を表す複数の質問文から質問文生成規則を獲得するのと同時に，入力された質問文間の関係を言い換え規則としても獲得する．学習時には質問表現辞書を参照する．

学習データ，質問表現辞書，ルール辞書に記述される内容は全て本システムが扱う意味構造で表現される．

4.3.1 学習データ

学習処理部に与えられる学習データは同じ内容を表す一つ以上の質問文から成り，言い換えや省略を含む質問文の組である．ただし，ある文に対して言い換えまたは省略を行なった際，異なる内容を表す質問文になってはいけない．

以下に例を示す．例には表層文も併記してあるが，実際の処理には用いられない．

(2)

・[[main 帰ってくる] [time いつ] [goal 札幌] [atb future]]
（いつ札幌に帰ってくる？）

・[[main 戻ってくる] [time いつ] [goal 札幌] [atb future]]
（いつ札幌に戻ってくる？）

・[[main 教えてください] [obj [[main 日にち]
[mod [[main 帰ってくる] [goal 札幌] [atb future]]]]]
(札幌に帰ってくる日にちを教えてください)

(3)

・[[main 帰ってくる] [time いつ] [agent 太郎]
[goal 札幌] [atb future]]
（いつ太郎は札幌に帰ってくる？）

・[[main 帰ってくる] [time いつ] [goal 札幌] [atb future]]
(いつ札幌に帰ってくる？)

(4)
・[[main 帰ってくる] [time いつ] [goal 札幌] [atb future]]
(いつ札幌に帰ってくる？)
・[[main 戻ってくる] [time いつ] [goal 札幌] [atb future]]
(いつ札幌に戻ってくる？)
・[[main 帰ってくる] [time いつ] [agent 太郎]
[goal 札幌] [atb future]]
(いつ太郎は札幌に帰ってくる？)
・[[main 教えてください] [obj [[main 日にち]
[mod [[main 帰ってくる] [goal 札幌] [atb future]]]]]
(札幌に帰ってくる日にちを教えてください)

(5)
・[[main 帰ってくる] [time いつ] [goal 札幌] [atb future]]
(いつ札幌に帰ってくるか？)
・[[main 戻ってくる] [time いつ] [goal 東京] [atb future]]
(いつ東京に戻ってくるか？)

学習データは(2)のような言い換え関係として与えてもよいし，(3)のような省略関係でも良い．また，(4)のように(2)と(3)を同時に学習データにすることができる．しかし，(5)のような「いつ～帰ってくる」と「いつ～戻ってくる」の言い換え関係があっても文全体が異なる内容を表す場合はこのような学習データをシステムに入力してはいけない．これは，本システムが与えられたデータの共通部に注目して一般化された規則を獲得するためであり，(5)のように一般化されるべき場所に異なる語があると一般化することができないためである．

4.3.2 学習処理

学習データと質問表現辞書を用いて規則を獲得する．学習データは意味表現で与えられ，学習データ内の質問文同士で共通のスロット値があれば，これを変数に置き換える．このとき使用される変数は@1，@2のように"@＋数字"で表現され，学習データ内の同じ語は同じ変数に置き換わる．

ただし，変数に置き換える際，質問表現辞書に記述されている質問表現と同一の構造があれば，その構造部分に含まれる語は一般化処理の対象にはならない．また，一般化の対象となる語がデータ内に含まれる質問文のmainである場合にも，一般化の処理は行なわないことにしている．これは，mainまで一般化してしまうと言い換え関係の情報が失われてしまう場合があることや，ルールの適用範囲が飛躍的に広がってしまい適用を誤る回数が多くなることが予想されるからである．例えば「いつ札幌から帰るか？」「いつ帰るか？」「いつ戻るか？」という３データから学習を行なったとき，共通部の「帰る」を一般化してしまうと「戻る」との言い換え情報を失ってしまう．

次に学習例を示す．

(6)
・[[main 帰ってくる][time @0][goal @1][atb @2]]
(@0@1 に帰ってくるか？)
・[[main 帰ってくる][time @0][agent @3]
[goal @1][atb @2]]
(@0@3 は@1 に帰ってくるか？)
・[[main 戻ってくる][time @0][agent @3]
[goal @1][atb @2]]
(@0@3 は@1 に戻ってくるか？)
・[[main 教えてください][obj [[main 日にち]
[mod [[main 帰ってくる][goal @1][atb @2]]]]]]
(@1 に帰ってくる日にちを教えてください)

前節の(4)を学習データとして規則獲得を行なった．各共通部が変数化されている．ただし，文の主 main である「帰ってくる」「戻ってくる」「教えてください」は一般化の対象にならない．ここで，質問表現辞書に[time いつ]が存在したとき(6)の[time @0]は一般化されないが，ここでは辞書に[time いつ]が存在しないので(6)のように規則が獲得され，この後フィードバック部で[time いつ]へと修正されることになる．

学習処理部で生成された結果は一つのルールとしてルール辞書に記述される．つまり一つの学習データから一つの関係規則が生成されることになる．

4.4 ルール辞書

質問文生成規則及び言い換え規則が記述される．ルール辞書では学習処理時にルールが追加されるほかに，フィードバック時にもルールの更新が行なわれる．

学習処理部で一般化された学習データは一つのルールとしてまとめ，そのルールに含まれる質問文は互いに言い換え関係や省略関係を持つことになる．また，学習時に過度に一般化された質問文の変数は，フィードバック時に特殊化が行なわれ元の値に戻るため，ルール辞書には一般化された規則だけではなく変数化される前の語も共に記述される．

(7)
0-1)[[main 帰ってくる][time @0][goal @1][atb @2]]
0-2)[[main 帰ってくる][time @0][agent @3]
[goal @1][atb @2]]
0-3)[[main 戻ってくる][time @0][agent @3]
[goal @1][atb @2]]
0-4)[[main 教えてください][obj [[main 日にち]

[mod [[main 帰ってくる][goal @1][atb @2]]]]]]
@0：いつ，@1：札幌，@2：future，@3：太郎

0-3)はルール辞書の 0 番目のルールの 3 番目のデータを示す．ルール 0 には四つの質問文があり，0-1)から 0-4)は一つの言い換え規則として表現される．

4.5 応答生成部

ユーザが入力した文に対して，ルール辞書のルールを用いてシステムが質問文を生成する．入力は状況を表す文を意味表現に変換したものを扱う．ここで扱う状況を表す文とは「太郎が東京に帰ってきた」「会議が 3 時に始まる」というような，ある状況を表す内容の文のことを指す．

ユーザから状況を表す文が入力されると，次にその文と同様の構造を持つ質問文を含むルールが選択される．ここで述べている同様の構造を持つ質問文とは，入力文と質問文の main が一致し，かつ，入力文と質問文が同一スロットを持つ場合そのスロット値が全て同じ値である場合をいう．選択された質問文と入力文の構造を比較することで，選択したルールの変数部に入力文から抽出された語が代入される．

選択されたルールの各変数に入力文の語を代入した後，ルール内の各質問文が出力される．選択されたルールのいずれかの質問文にまだ変数が残っている場合に，そのルールが質問表現を含むときはその質問文は生成しない．一方，質問表現を含まないときはその質問文の変数を変数に置き換える前の語に戻して出力する．これを特殊化とよぶ．

このような処理を行なう理由として，変数が残っている質問文においてその質問文に質問表現が既に含まれているのであれば，残された変数は単に情報が不足しているために語が代入されなかった結果と考えることができるので，本システムではそのような質問文を生成していない．また，質問文に質問表現が含まれない場合には，残された変数は一般化すべきではない語（ここでは質問表現）を過度に一般化したために生じた可能性があると考えることができるので，変数を特殊化して質問文を生成することにしている．

次に(8)を入力文として(7)のルール 0 が適用された例を(9)に，ルール 0 の[time @0]を[time いつ]に置き換えたルールが適用された例を(10)にそれぞれ示す．

(8)
入力) [[main 帰ってくる][goal 東京][atb past]]
(東京に帰ってきた)
(9)
0-1)[[main 帰ってくる][time いつ][goal 東京][atb past]]
(いつ東京に帰ってきた？)
0-2)[[main 帰ってくる][time いつ][agent 太郎]
[goal 東京][atb past]]
(いつ太郎は東京に帰ってきた？)
0-3)[[main 戻ってくる][time いつ][agent 太郎]
[goal 東京][atb past]]
(いつ太郎は東京に戻ってきた？)
0-4)[[main 教えてください][obj [[main 日にち]
[mod [[main 帰ってくる][goal 東京][atb past]]]]]]
(東京に帰ってきた日にちを教えてください)
(10)
0-1)[[main 帰ってくる][time いつ][goal 東京][atb past]]
0-4)[[main 教えてください][obj [[main 日にち]
[mod [[main 帰ってくる][goal 東京][atb past]]]]]]

(9)では，全ての質問文に質問表現が含まれていないので@0 と@3 がそれぞれ「いつ」と「太郎」（二重下線部）に特殊化されている．(10)では 0-1)と 0-2）に@3 が残ったが，質問表現[time いつ]が含まれるので特殊化は行なわれずに質問文は生成されていない．

4.6 フィードバック部

応答生成部で出力された結果の正誤をユーザが判定する．判定基準はユーザが入力した状況に対してシステムが適切な質問文を生成するか否かである．与えられた状況よりも情報が省略されて質問文が生成される場合は適切であるが，与えられた状況以外の情報が付加されている場合は不適切とみなす．

前節で述べた(9)の結果を例にとると，0-1),0-4)は「東京に帰ってきた」という入力に対して適切な質問を行なっていると考えることができるが，0-2),0-3)は入力からは得ることのできない「太郎」という情報が付加されているので適切な応答とはいえない．このことから 0-1),0-4)は”正”，0-2),0-3)は”誤”となる．

以上のような判定をユーザが行い，その結果をがフィードッバック部に入力されることになる．フィードバック部に”正”が入力されると，対象となったルールの変数が特殊化されている場合には，その語を質問表現とみなして質問表現辞書への登録が行なわれる．また，ここで変数を特殊化した質問文を新たな規則としてルール辞書の更新を行なう．

(9)の 0-1)の質問文を”正”としてフィードバックすると，[time いつ]が質問表現として質問表現辞書に登録され，

(11)
0-1)[[main 帰ってくる][time いつ][goal @1][atb @2]]

という質問文が新たにルール辞書に登録される．

5. 評価実験

5.1 実験内容

時間を尋ねる同じ内容を表す質問文から成る組 14 組（合計質問文数 44）とその組に対応する評価データを用意し，質問文生成規則の学習および評価データによるシステムの稼動実験を行なった．学習データに即した任意の状況を表す文を意味構造に変換し，これを評価データとして与えシステムがどのような質問を生成できるかを観察する．同時に，学習時のルール獲得状況についても考察した．実験に用いた学習データ及び評価データは第一著者が作成している．

ここで，学習データを時間を表す質問表現のみに限定した理由として，一つの質問内容に関して質問を生成することで実験結果の検証が行ないやすくなることや，実験で時間に関する質問規則を獲得できることがわかれば，本システムが意味表現の空きスロットを埋めて質問を行なうという操作が基本であることを考えると，他の質問に対しても同様の操作で規則が獲得されるので，他の質問表現でも同様の結果が期待できると予想されるからである．

また，評価データを学習データに即した状況を表すデータのみに限定している理由として，本システムが獲得する質問文生成規則では，意味構造の **main** スロットの値が一般化されることがないため，入力データの **main** スロットの値が規則の **main** スロットの値と異なる場合には，単純にルールの適用率が下がるだけであり，本システムの検証を行うには相応しくないと考えた．また，学習データをさらに追加することによりこの問題は解消されると予想されるので，本実験ではこれらのデータのみで実験を行った．

この実験では，学習データおよびシステムへの入力・出力は全て表層表現を 3 章で述べた意味表現形式に変換したものを使用している．将来的には，表層文から自動的に意味構造へ変換しシステムが処理を行うように改良する予定であるが[9],[11],[12]，現段階では意味解析により処理精度が低下することも含め，表層表現を直接扱うのではなく，3.1 で述べた意味表現を用いてシステムを運用している．

なお，実験はルール辞書および質問表現辞書を空の状態から始め，生成された質問文の評価は第一著者が評価している．

5.2 実験結果

実験を行なった結果，ルール獲得状況及び応答生成状況の両方の側面から実験結果をまとめると大きく次のように分類できた．括弧内の数字は対象となる質問文の数（合計 44）である．

(a)適切な質問(26).
(b)学習不足のため一般化が行なわれない質問(11).
(c)過度に特殊化を行なっている(4).
(d)適切な質問を生成できる規則だが，本システムでは適用できない(3).

それぞれの例を次に示す．

(a)適切な質問
入力) [[main 帰ってくる][goal 東京][atb past]]
出力 1)[[main 帰ってくる][time いつ][goal 東京][atb past]]
出力 2)[[main 教えてください][obj [[main 日にち]
[mod [[main 帰ってくる][goal 東京][atb past]]]]]]

(b)学習データ不足のため一般化が行なわれていない
入力) [[main 帰ってくる][atb past]]
規則)[[main 帰ってくる][time いつ][agent 太郎] [atb @0]]
出力)[[main 帰ってくる][time いつ][agent 太郎] [atb past]]

(c)過度に特殊化を行なっている
入力) [[main 帰ってくる][agent 太郎][atb past]]
規則)[[main 教えてください][obj [[main @0]
[mod [[main @1] [goal @2] [atb @3]]]]]]
出力)[[main 教えてください][obj [[main 時間]
[mod [[main 帰ってくる][goal 東京][atb past]]]]]]

(d)適切な質問をできる規則はあるが適用できない
入力)[[main いる][place 駅][agent 太郎]]
規則)[[main いる][place [[main どこ][mod @0]]]]
出力）生成不可

6. 考察

実験の結果，適切な応答が 26 文生成された．最初は適切な応答が生成されない場合でもフィードバックによりルールの改善が行なわれ正しい結果が生成されることも確認された．このことから，本手法は言い換え規則と質問生成規則を同時に効率的に獲得していると考えることができ，フィードバック処理によりルールの精度を向上させていることが確認された．

また，ルール獲得の際に過度に一般化が行なわれ，質問生成時に変数が残ってしまうような質問文が生成されたとしても，言い換えデータを用いることでシステムは適切な質問を行なうことができ，言い換え規則を獲得することによりシステムの質問生成能力が向上していることも確認されている．

しかし一方で，適切な質問が生成されなかった場合も確認された．原因を考察してみると，学習処理部もしくは応答生成部によって5.2の(b),(c)のような結果が得られる場合と，本システムの質問文生成規則獲得手法によって 5.2 の(d)のような結果が生成される場合

の三点が大きく考えられる．

まず最初に，学習処理部によって 5.2 の(b)のような結果が得られる場合であるが，本システムの学習処理部では与えられた学習データの共通部を一般化しているので，共通な部分があればその構造は一般化され，共通な部分がなければ当然一般化されない．このとき，本来一般化されるべき語が複数あればその語は一般化されることになるが，複数含まれなければ一般化されることはない．そのため一般化されない語（ここでは「太郎」)を含む誤った質問文が生成されたり，語が一般化されていないためにルールが適用できず質問文が生成されない結果が見られた．また，適切な質問が生成されない場合の中で最も割合が多かった理由として，本システムの学習処理による一般化の効率が悪いことが考えら得る．

次に応答生成部によって 5.2 の(c)のような結果が生成された場合であるが，本システムでは応答生成時に変数が残ったデータに質問表現が含まれない場合は，その変数を質問表現と仮定し変数を特殊化して質問文を生成する．このとき，対象となるデータに特殊化すべき変数(@0,@1)と一般化したままであるべき変数(@2)の両方が含まれると誤った質問文を生成してしまう．学習データ内のほかの質問文との比較，または他のルールで獲得された質問表現を参照することで，一方の変数が特殊化されもう一方が変数のまま残る場合もあるが，与えられた学習データによっては正しい特殊化および一般化が行なわれない場合も存在する．

本システムが一つの学習データから一つのルールのみを生成することや前述したルール獲得時の問題点を合わせて考えると，学習データ内の言い換え規則の数が獲得ルールの精度に強い影響を与えることがわかった．これを改善するには獲得したルール同士が互いに参照しあうことによって規則を更新したり，応答生成時にも与えられた情況からルールの一般化等を試みる必要があると考えられる．

最後に 5.2 の(ｄ)のような場合であるが，3.2 で述べたように本システムの基本となる質問生成処理は空きスロットを埋めて質問を行なう処理なので，既にそのスロットが埋まっていると質問を生成することができない．多くの場合はこれでも適切に質問が生成されるが，中にはルール自体は適切なのに与えられた状況に適用できない場合も観察された（(d)の規則の@0 に「駅」を代入して質問文を生成できない)．これは単に空きスロットを管理するだけでは全ての質問を行うことができないことを示しており，この点を改良する必要がある．ただしこの質問が生成できなくてもルール内に生成可能な言い換えが存在すればその文が生成されるのでシステムは質問を行なうことができる．

また，今回の実験では学習データの数が十分とは言えず，実験からは観察できなかった問題点があることも予想される．この点については，今後学習データを増やしさらなる検証を行なっていく予定である．

7. おわりに

本稿では帰納的学習により複数の質問データから言い換え規則と質問生成規則を獲得する手法を提案し評価実験を行なった．実験の結果，多くの場合に適切な質問を生成できることが確認された．一方で，質問生成の精度は学習データの内容に大きく影響されることや，本手法のみでは学習した全ての質問を生成することができないこともわかり，この点は改良する必要性がある．今後，学習データを増やし，さらなる考察を行っていきたい．

また，本稿では与えられた状況からシステムが質問を生成する場合のみを対象としたが，今後ユーザとの対話から状況を導き質問を行なう一連の処理を行なえるようにシステムを拡張していく予定である．

文　献

[1] 石川由紀子，加藤恒昭，“質問表現生成のための発話内容決定機構－質問内容の特徴に基づく発話内容の決定”，人工知能学会誌，Vol.10, No.6, pp.962-970, 2000.

[2] 村山耕一，荒木健治，栃内香次，“帰納的学習を用いた質問応答例からの質問表現生成規則の獲得”，情報処理学会研究会報告，Vol.2001, No.9, pp.63-68, 2001.

[3] Weizenbaun,J., ELIZA - A Computer Program for the Study of Natural Language Communication Between Man And Machine, Communications of the Asociation for Computing Machinery, vol.9, No.1, pp.36-45, 1996.

[4] Winograd, T. , “Understanding Natural Language”, Academic Press, 1972.

[5] ジョン・バーワイズ，ジョン・ペリー，土屋俊，鈴木浩之，白井英俊（訳)，片桐恭弘，向井国昭，“状況と態度”，産業図書，1992.

[6] 高橋哲郎，岩倉友哉，飯田龍，乾健太郎，“Kura：統一的かつ宣言的記述法に基づく言い換え知識の開発環境”，信学技報，TL2001-12, pp.23-30, 2001.

[7] 村田真樹，井佐原均，“同義テキストの照合に基づくパラフレーズに関する知識の自動獲得”，Vol.2001, No.20, pp.129-136, 2001.

[8] 村山耕一，荒木健治，栃内香次，“帰納的学習による意味表現を用いた質問文生成ルールの獲得”，情報処理北海道シンポジウム 2000 講演論文集，pp.54-57,2000.

[9] 峨家正樹，荒木健治，栃内香次，“帰納的学習を用いた構文構造から意味構造への変換手法とその評価”，信学技報，NLC2000-80, pp.41-48, 2001

[10] 日本電子化辞書研究所，“EDR 電子化辞書仕様説明書”,1995.

[11] 荒木健治，栃内香次，“帰納的学習による語の獲得および確実性を用いた語の認識”，電子情報通信学会論文誌 D-II, Vol.75-D-ii, No.7, pp.1213-1221, 1992.

[12] Y. Masatomi, K. Araki, and K. Tochinai, “A SYNTACTIC ANALYSIS METHOD USING INDUCTIVE LEARNING”, a pp.447-451, 2000.